IEEE830A

3.4 Restricciones de Diseño

Esta sección describe todas aquellas limitaciones que condicionan las decisiones de diseño e implementación del SGRSI, independientemente de los requisitos funcionales. Su cumplimiento es obligatorio y no negociable en ninguna etapa del desarrollo.

Restricciones Tecnológicas del Stack:
El sistema se construirá únicamente sobre la base tecnológica establecida por la institución. Esto implica usar HTML5 semántico con CSS3, JavaScript nativo (*vanilla*) para resolver la interactividad en el cliente, PHP estructurado bajo el paradigma de Programación Orientada a Objetos (POO) en el *backend*, y MySQL para la gestión de la base de datos relacional. Queda descartado el uso de cualquier *framework* tanto de *backend* (como Laravel o Symfony) como de *frontend* (como React o Vue), excepto que se cuente con el aval explícito de los docentes.

Arquitectura en Tres Capas:
La aplicación debe estructurarse garantizando una división clara y rigurosa entre la capa de presentación, la capa de lógica de negocio y la capa de acceso a datos. Este acoplamiento define por completo la organización de las carpetas del proyecto, la forma de nombrar las clases y el reparto de tareas entre los distintos componentes del código.

Normalización de la Base de Datos:
El diseño del modelo relacional debe alcanzar el estándar de la Tercera Forma Normal (3FN), respaldado por su correspondiente documentación en DDL y DML. Bajo ninguna circunstancia se admitirán dependencias transitivas ni datos redundantes que pongan en riesgo la integridad referencial de las tablas.

Seguridad en el Acceso a Datos:
Para blindar el sistema contra ataques de inyección SQL, todas las consultas a la base de datos se programarán utilizando consultas preparadas con parámetros vinculados. No se construirán sentencias SQL concatenando directamente las entradas del usuario, asegurando que ningún dato externo viaje hacia la base de datos sin ser previamente parametrizado.

Control de Acceso Basado en Roles (RBAC):
El sistema se estructurará sobre tres perfiles de usuario bien diferenciados: Administrador/Coordinador, Técnico/Soporte y Solicitante, asignando a cada uno permisos específicos. El proceso de autenticación protegerá las credenciales mediante un método de encriptación técnicamente justificado, priorizando el uso de `password_hash()` con el algoritmo `PASSWORD_BCRYPT`. Para garantizar una seguridad real, la validación de las sesiones y el

control de los permisos se procesarán siempre en el servidor, evitando delegar esta lógica únicamente en el navegador del cliente.

Internacionalización:
La interfaz de usuario estará preparada para dar soporte bilingüe en español e inglés. Esta condición influye directamente tanto en el desarrollo de las vistas como en la organización de los archivos de traducción, por lo que su integración debe planificarse desde el diseño de la arquitectura inicial.

Despliegue con Contenedores: El entorno de ejecución de la aplicación se gestionará mediante *docker-compose*, dividiendo el sistema en al menos dos servicios independientes: uno destinado a la aplicación PHP y otro exclusivo para el servidor MySQL, este esquema de virtualización define de antemano la estructuración de las rutas de configuración, el manejo de las variables de entorno y el control de las dependencias.

Entorno de Red:
La plataforma está pensada para funcionar únicamente dentro de la intranet institucional. Al no estar expuesto al acceso desde redes públicas, las decisiones de seguridad perimetral y las políticas de configuración del servidor se adaptarán específicamente para responder a las necesidades de este entorno cerrado.

Restricciones de Estándares:
Toda la documentación técnica del proyecto seguirá estrictamente los lineamientos establecidos en el Anexo 1 del documento de pautas. Esto incluye el uso de tipografía Arial o Times New Roman (configurando los títulos en 16 pt, subtítulos en 14 pt y párrafos en 12 pt), márgenes estándar, un estilo de redacción impersonal y la organización de las referencias bibliográficas bajo las normas APA en su 7.ª edición.

3.5 Atributos del Sistema

Fiabilidad:
El sistema debe garantizar la integridad de los datos registrados en el historial, préstamos y en tickets, las operaciones de alta, baja y modificación de activos deben ejecutarse dentro de transacciones de base de datos para evitar estados inconsistentes ante fallos que puedan ocurrir. Se estima una disponibilidad objetivo del 99% durante el horario institucional (turnos matutino, vespertino y nocturno).

Mantenibilidad:
La arquitectura en tres capas y el paradigma POO facilitan la localización y corrección de defectos, cada clase debe tener responsabilidad única (principio SRP). El código debe incluir comentarios de bloque en métodos públicos con descripción de parámetros y valor de retorno. El sistema debe disponer de un manual de instalación y mantenimiento que permita a un técnico con conocimientos básicos de Linux y Docker restaurar el servicio ante una incidencia.

Portabilidad:
Al empaquetar el entorno con *docker-compose*, nos aseguramos de que el sistema funcione exactamente igual en cualquier computadora, independientemente del sistema operativo que tenga instalado. Para que esto sea 100% reproducible y evitar sorpresas, es obligatorio definir la imagen base del contenedor de la aplicación con una versión fija y específica (por ejemplo, php:8.2-apache).

Seguridad:

- **Autenticación:** Se implementará un formulario de inicio de sesión, con validación servidor, las contraseñas serán guardadas como hash irreversible. Las sesiones PHP deben regenerarse tras el logeo exitoso del usuario, para evitar un ataque de fijación de sesión.
- **Autorización**: El sistema verificará el rol del usuario antes de permitir el acceso a cualquier función o módulo, Administrador/Coordinador, Técnico/Soporte y Solicitante, solo podrá acceder a las opciones autorizadas para su rol, impidiendo el acceso a funcionalidades no permitidas.
    - **Integridad de entrada:** Bajo el principio de defensa en profundidad, los datos que ingrese el usuario se validarán y sanearán por duplicado: primero en el cliente con JavaScript (para mejorar la experiencia de usuario) y obligatoriamente en el servidor con PHP (para garantizar la seguridad real).
- **Protección contra CSRF**: Todos los formularios que realicen cambios de estado o modifiquen información en el sistema incluirán un token CSRF único y dinámico, vinculado a la sesión del usuario, para prevenir peticiones maliciosas forzadas.
- **Auditoría:** Para mantener la trazabilidad de la plataforma, el sistema registrará automáticamente cualquier acción crítica (como el alta o baja de usuarios, la modificación de activos o los cambios de estado en los tickets). Cada registro de auditoría guardará el usuario responsable y la marca de tiempo exacta del evento.

Usabilidad:

- **Experiencia de Usuario:** Interfaz accesible y fácil de usar, adaptada para personal docente y administrativo con competencias digitales básicas.
- **Mensajes del Sistema:** Ante un fallo, el sistema mostrará alertas claras y orientadas a la acción del usuario final, omitiendo tecnicismos del *backend*.
- **Rendimiento:** El tiempo máximo de respuesta y carga para cualquier pantalla se fija en 3 segundos bajo el entorno de red de la intranet.

Escalabilidad:
Aunque el sistema está creado para un entorno institucional con una carga de usuarios moderada (detallada en el punto 3.2), su arquitectura está preparada para el futuro, gracias a la separación estricta en tres capas y al despliegue mediante contenedores, la plataforma

permite realizar un escalamiento horizontal del servicio de aplicación de manera sencilla y sin necesidad de modificar una sola línea del código.

3.6 Otros Requisitos

Gestión de Respaldos:
La plataforma contará con una estrategia de *backup* automatizada mediante *scripts* en Bash diseñados de forma modular. Estos se encargarán de realizar copias de seguridad periódicas de la base de datos MySQL, las cuales se almacenarán en una ubicación independiente del servidor principal para garantizar la disponibilidad de la información y permitir la verificación de su integridad.

Repositorio Git:
El código se gestionará en un repositorio alojado bajo el nombre de la empresa ficticia asignada al grupo. Se trabajará con una rama principal (main) y ramas específicas por funcionalidad (*feature branches*), aplicando convenciones claras para los mensajes de *commit* y utilizando etiquetas de versión (v1.0, v2.0, v3.0) para identificar de manera ordenada cada una de las entregas.

Compatibilidad de navegadores:
El sistema debe ser funcional en las versiones vigentes de los navegadores Firefox, Chrome y Edge, sin requerir plugins adicionales.

4 Apéndices

Módulo/Funcionalidad	Solicitante	Técnico/Soporte	Administrador/Coordinador
Crear ticket de incidencia	✅	✅	✅
Atender y actualizar ticket	❌	✅	✅
Consultar estado de ticket propio	✅	✅	✅
Registrar préstamo/devolución de equipo	❌	✅	✅
Consultar inventario	❌	✅	✅
ABM de activos tecnológicos	❌	❌	✅
Crear/gestionar usuarios	❌	❌	✅
Acceder al dashboard de métricas	❌	❌	✅

Módulo/Funcionalidad	Solicitante	Técnico/Soporte	Administrador/Coordinador
Solicitar servicio (laboratorio/software)	✅	✅	✅
Registrar diagnóstico/resolución	❌	✅	✅

4.B Formatos de entrada/salida principales

Formulario de alta de activo tecnológico (entrada):
Para registrar un nuevo equipo, se deberá ingresar su número de serie, una descripción detallada y seleccionar su categoría (equipo, periférico o componente) y su estado actual (operativo, en reparación o de baja). También se debe especificar su ubicación física, la fecha en la que se incorpora al sistema y los datos del usuario que está realizando el registro.

Formulario de creación de ticket (entrada):
Para reportar un problema, el usuario tendrá que escribir un título y explicar la situación. Si es necesario, podrá poner el equipo que está fallando y sugerir un nivel de prioridad. la fecha, la hora y los datos del solicitante, no se piden de forma manual, sino que el sistema los toma automáticamente de la sesión activa.

Reporte de métricas (salida): Este informe presentará de forma clara los equipos que fallan más seguido, cuánto tarda en promedio cada técnico en resolver un problema, la cantidad de solicitudes que entran en un período de tiempo y la relación entre los tickets resueltos y los que siguen pendientes. Toda esta información se mostrará en una tabla HTML en la pantalla, con un botón que permita exportar los datos.